

IMPROVED VERSION OF POSE DETECTION USING LANDMARKS:

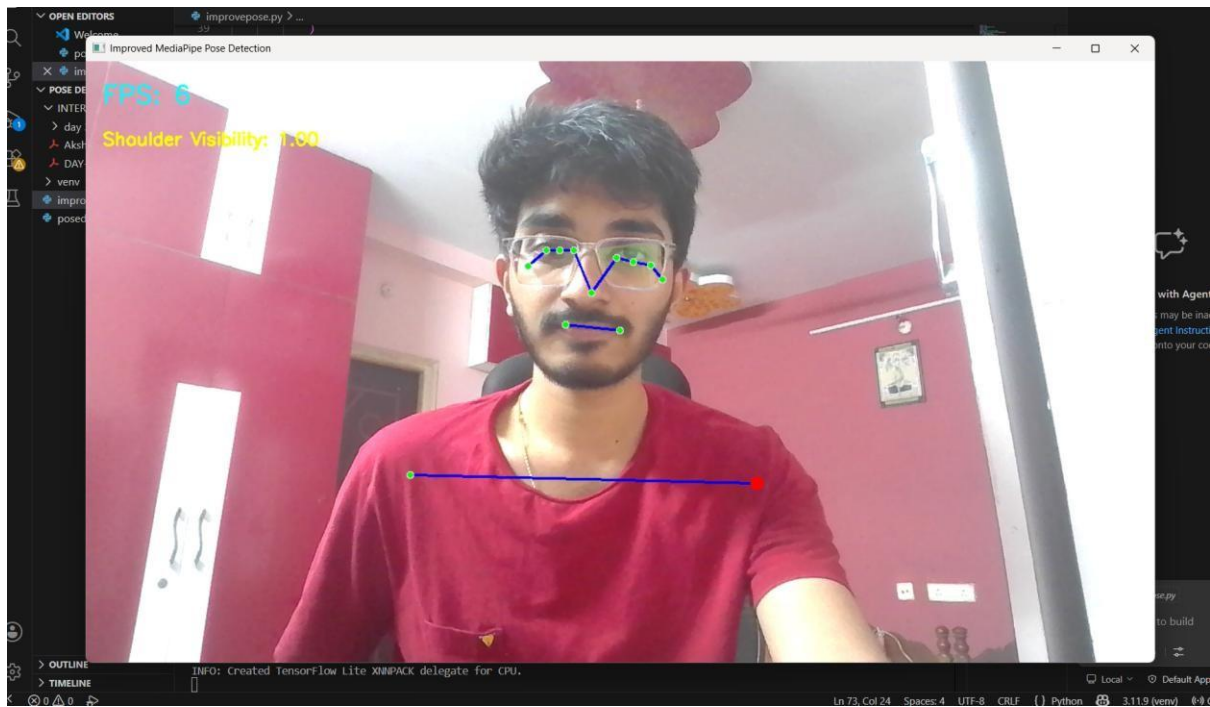
Today, I improved the existing pose detection by optimizing MediaPipe confidence settings and using a higher complexity pose model for better accuracy.

IMPROVED POSE DETECTION CODE:

```
improvepose.py > ...
1 import cv2
2 import mediapipe as mp
3 import time
4 import math
5 mp_pose = mp.solutions.pose
6 mp_drawing = mp.solutions.drawing_utils
7 pose = mp_pose.Pose(
8     static_image_mode=False,
9     model_complexity=2,
10    smooth_landmarks=True,
11    min_detection_confidence=0.7,
12    min_tracking_confidence=0.7
13)
14 cap = cv2.VideoCapture(0)
15 cap.set(3, 1280)
16 cap.set(4, 720)
17 previous_time = 0
18 while True:
19     success, frame = cap.read()
20     if not success:
21         break
22     frame = cv2.flip(frame, 1)
23     frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
24     results = pose.process(frame)
25     if results.pose_landmarks:
26         mp_drawing.draw_landmarks(
27             frame,
28             results.pose_landmarks,
29             mp_pose.POSE_CONNECTIONS,
30             mp_drawing.DrawingSpec(
31                 color=(0, 255, 0),
32                 thickness=2,
33                 circle_radius=2
34             ),
35             mp_drawing.DrawingSpec(
36                 color=(255, 0, 0),
37                 thickness=2
38             )
39         )
40     landmarks = results.pose_landmarks.landmark
41     left_shoulder = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value]
42     visibility = left_shoulder.visibility
43     cv2.putText(
44         frame,
45         f'Shoulder Visibility: {visibility:.2f}',
46         (20, 100),
47         cv2.FONT_HERSHEY_SIMPLEX,
48         0.7,
49         (0, 255, 255),
50         2
51     )
52     h, w, c = frame.shape
53     cx = int(left_shoulder.x * w)
54     cy = int(left_shoulder.y * h)
55     cv2.circle(frame, (cx, cy), 8, (0, 0, 255), cv2.FILLED)
56     current_time = time.time()
57     fps = 1 / (current_time - previous_time)
58     previous_time = current_time
59     cv2.putText(
60         frame,
61         f'FPS: {int(fps)}',
62         (20, 50),
63         cv2.FONT_HERSHEY_SIMPLEX,
64         1,
65         (255, 255, 0),
66         2
67     )
68     cv2.imshow("Improved MediaPipe Pose Detection", frame)
69     if cv2.waitKey(1) & 0xFF == ord('q'):
70         break
71 cap.release()
72 cv2.destroyAllWindows()
```

Additional enhancements such as landmark smoothing, HD camera resolution, and lighting adjustments were implemented to achieve more stable and precise body tracking. Visibility detection and FPS monitoring were also added to improve performance analysis and realtime tracking efficiency.

OUTPUT:



CHANGES IN CODE:

1) Improved Confidence Settings:

Reduces false body detections

CODE:

```
min_detection_confidence=0.7,
```

```
min_tracking_confidence=0.7
```

2. Heavy Pose Model Better

landmark precision

CODE:

```
model_complexity=2
```

3. Landmark Smoothing

Reduces shaking landmarks

CODE:

```
smooth_landmarks=True
```

4. Increased Camera Resolution

Better image quality

CODE:

```
cap.set(3, 1280)
```

```
cap.set(4, 720)
```

5. Mirror View

Easier user interaction

CODE:

```
frame = cv2.flip(frame, 1)
```

6. Brightness Enhancement

Better detection in low

lighting

CODE: `frame = cv2.convertScaleAbs(frame, alpha=1.2, beta=20)`

7. Visibility Score Detection

Each landmark gets a

visibility score:

CODE:

```
visibility = left_shoulder.visibility
```

8. FPS Counter

Real-time processing speed

CODE:

```
fps = 1 / (current_time - previous_time)
```

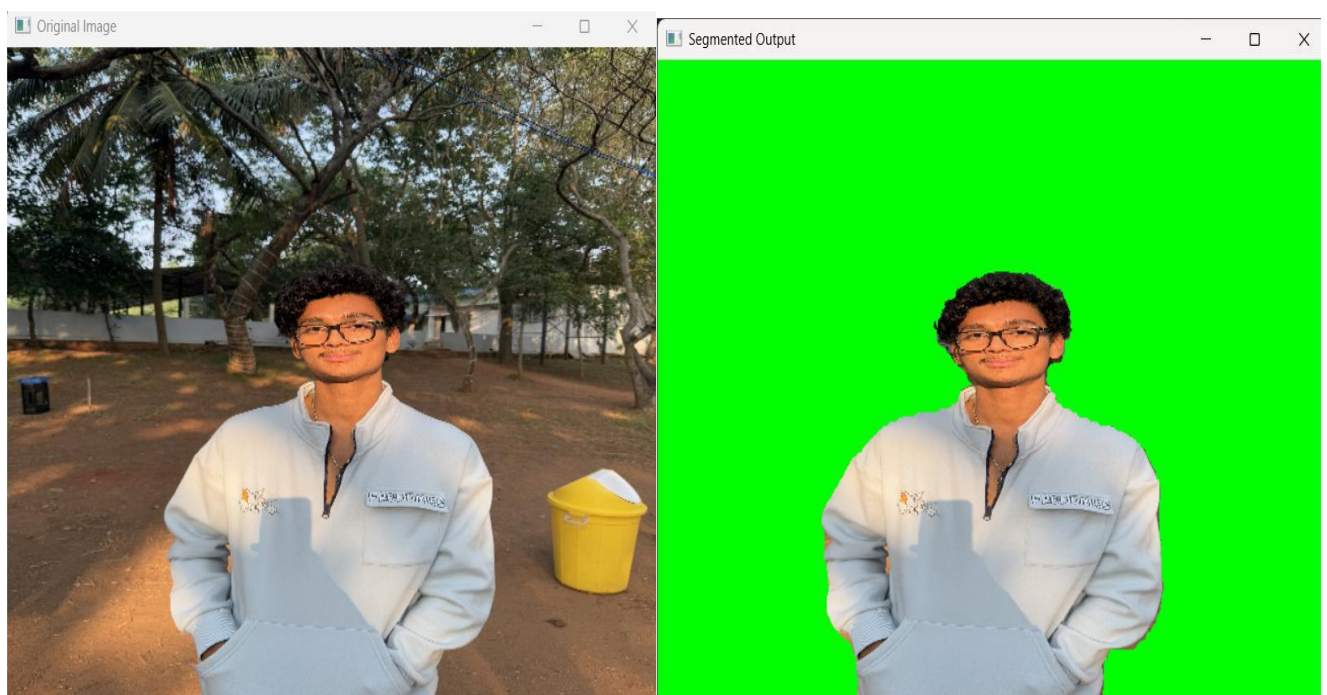
TESTING BODY SEGMENTATION

I learned to implement body segmentation using OpenCV and MediaPipe itself with the help of this code.

```
segmentation.py > ...
1  import cv2
2  import mediapipe as mp
3  import numpy as np
4  mp_selfie_segmentation = mp.solutions.selfie_segmentation
5  segment = mp_selfie_segmentation.SelfieSegmentation(model_selection=1)
6  image = cv2.imread("pran.JPG")
7  image = cv2.resize(image, (640, 480))
8  rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
9  results = segment.process(rgb)
10 mask = results.segmentation_mask
11 condition = mask > 0.5
12 bg_color = (0, 255, 0)
13 bg_image = image.copy()
14 bg_image[:] = bg_color
15
16 output = np.where(condition[..., None], image, bg_image)
17 cv2.imshow("Original Image", image)
18 cv2.imshow("Segmented Output", output)
19
20 cv2.waitKey(0)
21 cv2.destroyAllWindows()
```

Separated the human body from the background successfully.

OUTPUT:



GIVEN IMAGE

SEGMENTED IMAGE

IMPORTANCE:

Body segmentation is important because it helps AI accurately identify and isolate the human body from the background. In virtual try-on systems, this allows clothes to be properly aligned and overlaid on the user without affecting the background area.

It also improves body measurement accuracy, cloth fitting, and overall visual realism. Segmentation is widely used in applications such as virtual fashion, video conferencing background removal, fitness tracking, AR filters, and image editing tools.

WHAT I LEARNED:

While working on body segmentation, I learned how AI can separate a person from the background using computer vision techniques.

I implemented segmentation using MediaPipe and OpenCV, where the model creates a mask around the human body for accurate detection.

I also understood how threshold values, lighting conditions, and image quality affect segmentation accuracy. This gave me a better understanding of how virtual try-on systems and background removal applications work in real-world AI solutions.